ĐẠI HỌC QUỐC GIA HÀ NỘI — TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN (VNU-HUS)
KHOA TOÁN - CƠ - TIN HỌC

BÁO CÁO BÀI TẬP CÁ NHÂN 3

AI-assisted Software Development & Software Testing & Quality Engineering

Học phần: Phát triển phần mềm với AI & Kỹ nghệ Kiểm thử Phần mềm (SE2026)

Họ và tên sinh viên:	Hà Minh Quang
Mã số sinh viên:	**23001916**
Mã biến thể cá nhân:	**VT-017** (Nhóm SE2026-T05 · Vai trò: Member B)
Đề tài dự án:	AI-05 – Vietnamese Meeting Minutes Assistant
Repository chính thức:	https://github.com/TAINGUYENVAN0507/SE2026_Codex
Baseline Commit Hash:	`2dea2bfdfea220d0492cedbd56d30340ed41f0b6`
Feature Commit Hash:	`30a93264c9a87d60e7f4c000bf689363578051aa`
Nhánh Git / Pull Request:	`feat/vt-017-stable-search-pagination`

CAM KẾT DANH DỰ: "Tôi chịu trách nhiệm về toàn bộ code, test và nội dung bài làm; tôi đã khai báo việc sử dụng AI."

Tóm tắt Biến thể Cá nhân hóa VT-017

- **Change request riêng:** Thêm tính năng tìm kiếm có lọc, phân trang và thứ tự kết quả ổn định vào endpoint `GET /api/v1/meetings`.
- **Risk trọng tâm:** Schema/API thay đổi làm client cũ bị hỏng (Breaking schema changes on legacy clients).
- **Kỹ thuật test bắt buộc:** Phân tích giá trị biên (Boundary Value Analysis - BVA).
- **Mutation / Deliberate bug bắt buộc:** Bỏ kiểm tra authorization (owner scoping) ở một đường đi tìm kiếm.
- **Mức tải kiểm chứng:** 100 người dùng/tác vụ đồng thời; mục tiêu p95 không quá 3.0 giây.
- **Ràng buộc làm việc với AI:** Patch nhỏ nhất; không đổi framework hoặc thêm dependency mới.

1. Specification và AI-assisted Plan

1.1. Problem Statement

Trong hệ thống **AI-05 (Meeting Minutes Assistant)**, endpoint danh sách cuộc họp `GET /api/v1/meetings` trước đây chỉ cung cấp tính năng lọc sơ sài theo một giá trị trạng thái (`status`) và tìm kiếm thô sơ trên tiêu đề, thiếu các bộ lọc thiết yếu như nguồn âm thanh (`source`), khoảng thời gian diễn ra cuộc họp (`from_date` – `to_date`). Nghiêm trọng hơn, cơ chế phân trang hiện tại sử dụng câu lệnh `order_by(Meeting.created_at.desc())` mà **không có secondary tie-breaker duy nhất** (như khóa chính `id`). Trong hệ quản trị cơ sở dữ liệu (PostgreSQL / SQLite), khi nhiều bản ghi có cùng mốc thời gian tạo `created_at` (do import hàng loạt hoặc tạo gần nhau), thứ tự trả về giữa các trang phân trang bị bất định (**non-deterministic pagination drift**), dẫn đến hiện tượng trùng lặp hoặc bỏ sót cuộc họp khi người dùng chuyển trang. Thách thức lớn nhất là việc mở rộng tính năng này tiềm ẩn **rủi ro phá vỡ schema (Core Risk)**: các client cũ (giao diện web frontend hoặc các ứng dụng bên thứ ba sử dụng model Pydantic kế thừa với `extra="forbid"`) sẽ bị crash ngay lập tức nếu API thay đổi cấu trúc trả về hoặc yêu cầu thêm các tham số bắt buộc.

1.2. Acceptance Criteria (AC)

- **AC-1 (Tìm kiếm & Lọc kết hợp):** Người dùng có thể tìm kiếm theo từ khóa `q` (so khớp không phân biệt hoa thường, tự động cắt khoảng trắng thừa, escape an toàn các ký tự đặc biệt `%`, `_`) và lọc đồng thời theo `status`, nguồn âm thanh `source` (`upload` | `recording`), khoảng ngày họp `from_date` và `to_date` (định dạng `YYYY-MM-DD`). Các bộ lọc là tùy chọn độc lập.
- **AC-2 (Phân trang ổn định tuyệt đối - Stable Result Ordering):** Hỗ trợ phân trang qua `page` (≥ 1, mặc định 1) và `page_size` ($1..100$, mặc định 10). Kết quả sắp xếp theo tiêu chí (`sort_by`: `created_at`, `updated_at`, `title`, `meeting_date`, mặc định `created_at`; `order`: `desc` | `asc`, mặc định `desc`) và **bắt buộc phải có secondary tie-breaker là `Meeting.id`**. Tập hợp ID trả về giữa các trang kế tiếp nhau phải rời nhau hoàn toàn ($S_i \cap S_j = \varnothing$) và bao phủ trọn vẹn dữ liệu.

- **AC-3 (Bảo toàn Schema Invariant & Tương thích ngược Client cũ - Core Risk):** Mọi tham số mới trên query string đều có giá trị mặc định tương thích hoàn toàn với hành vi cũ. Cấu trúc JSON trả về giữ nguyên 100% schema `MeetingListOut` (`items`, `page`, `page_size`, `total`), không phát sinh thêm bất kỳ trường lạ nào ở root level, đảm bảo các client cũ cấu hình `model_config = ConfigDict(extra="forbid")` không gặp lỗi `ValidationError`.
- **AC-4 (Cô lập dữ liệu người dùng - Owner Scoping Invariant):** Người dùng chỉ có thể tìm kiếm, lọc và xem các cuộc họp do chính mình sở hữu (`Meeting.owner_id == current_user.id`). Tuyệt đối không để lộ dữ liệu của người dùng khác dù từ khóa tìm kiếm khớp chính xác.

1.3. Non-Functional Constraints

- **NFC-1 (Hiệu năng tải đồng thời):** Hệ thống phải chịu tải tối thiểu 100 người dùng/tác vụ tìm kiếm & phân trang đồng thời với độ trễ $p95 \leq 3.0$ giây trên tập dữ liệu mẫu.
- **NFC-2 (Kiến trúc & Giới hạn Thư viện):** Patch phải đạt kích thước tối thiểu, đặt logic trong Service Layer (`app/services/meeting_search.py`), không làm phình router `meetings.py`, không thay đổi framework (FastAPI/SQLAlchemy) và không thêm bất kỳ third-party dependency nào.

1.4. Invariants & Contracts Cần Bảo Vệ

Contract Invariant (Bảo vệ Client cũ): Payload trả về bắt buộc tuân thủ hợp đồng OpenAPI `MeetingListOut`:

$$\text{Schema} = \{\text{items} : [...], \text{page} : \text{int}, \text{page_size} : \text{int}, \text{total} : \text{int}\}$$

với `ConfigDict(extra="forbid")`. Không được đổi tên trường (ví dụ `page_size` → `limit`) hoặc thêm các trường như `has_next`, `total_pages`.

Security Invariant (Cô lập dữ liệu): Mọi câu lệnh truy vấn dữ liệu bắt buộc phải gắn tiền đề:

$$\forall m \in \text{Results} : m.\text{owner_id} == \text{current_user.id}$$

Mọi bộ lọc nghiệp vụ (`q`, `status`, `date`) chỉ được phép thu hẹp trên tập con dữ liệu đã được cô lập theo quyền sở hữu.

1.5. AI-assisted Plan & Phân tích Đánh giá (Review)

- **Prompt ban đầu:** *"Hãy lập kế hoạch thực hiện cho Change Request VT-017: thêm tính năng tìm kiếm có lọc, phân trang và thứ tự ổn định vào endpoint GET /api/v1/meetings. Chú ý risk schema thay đổi làm hỏng client cũ và tạo patch nhỏ nhất."*
- **Assumptions của AI:** AI giả định có thể viết trực tiếp logic query vào route handler `meetings.py`; thêm trường `total_pages` và `has_next` vào schema phản hồi; và chỉ cần `order_by(sort_by)`.
- **Review và Bác bỏ/Sửa đổi bằng Evidence thực tế từ Repository:**
  1. **Bác bỏ Điểm 1 (Vi phạm Ranh giới Kiến trúc Module):** AI đề xuất viết toàn bộ câu truy vấn vào `backend/app/api/v1/meetings.py`. **Bằng chứng bác bỏ:** File `meetings.py` hiện đã phình to tới 2.451 dòng (18 routes). Tài liệu phân công kiến trúc tại `tasks/2026-11-06/member-b.md` (dòng 42) nêu rõ quy tắc: *"Không thêm logic mới vào meetings.py. Logic mới đặt trong app/services/, router chỉ gọi service."* **Khắc phục:** Tách toàn bộ nghiệp vụ tìm kiếm, lọc và phân trang vào service mới `backend/app/services/meeting_search.py`.
  2. **Bác bỏ Điểm 2 (Vi phạm Rủi ro Trọng tâm - Làm gãy Client cũ):** AI đề xuất bổ sung các trường metadata tiện ích `total_pages`, `has_next`, `has_prev` vào response. **Bằng chứng bác bỏ:** Tại `backend/app/schemas/meeting.py:105`, class `MeetingListOut` được bảo vệ bởi `model_config = ConfigDict(extra="forbid")`. Khi client cũ hoặc test hợp đồng (`test_openapi_drift.py`) parse JSON trả về, các trường mới sẽ kích hoạt lỗi `pydantic_core.ValidationError` khiến client dừng hoạt động. **Khắc phục:** Giữ nguyên tuyệt đối schema `MeetingListOut`.
  3. **Sửa đổi Điểm 3 (Thiếu Tie-breaker gây mất ổn định phân trang):** AI chỉ đề xuất `order_by(sort_column)`. **Bằng chứng sửa đổi:** Khi sắp xếp theo `created_at` hoặc `meeting_date`, các bản ghi có cùng giá trị sẽ có thứ tự bất định trong SQL khi phân trang bằng `OFFSET ... LIMIT`. **Khắc phục:** Bắt buộc chèn secondary tie-breaker duy nhất `Meeting.id` (`order_by(primary_col.desc(), Meeting.id.desc())`).

2. Risk-based Test Strategy và Bảng Test Cases

2.1. Phân tầng Kiểm thử & Justified Exclusion

- **Unit Tests:** Kiểm thử logic service `search_meetings`: thuật toán escape wildcard SQL, xử lý khoảng trắng, ánh xạ cột sắp xếp hợp lệ, và áp dụng tie-breaker.
- **Integration / API Tests:** Kiểm thử thông qua HTTP TestClient với database SQLite in-memory: thẩm định quyền truy cập (authentication/authorization), kiểm thử giá trị biên (BVA) của query parameters, kiểm tra tương thích ngược của schema phản hồi, cô lập dữ liệu người dùng (owner scoping).
- **E2E Tests:** Kiểm thử chu trình: Tạo meeting → Tìm kiếm có lọc → Phân trang liên tục.
- **Justified Exclusion (Ngoại trừ hợp lý):** Không đưa pipeline trích xuất âm thanh (Whisper, PyAnnote) vào bộ test của change request này. Lý do: Tính năng tìm kiếm & phân trang metadata thuần túy thuộc tầng API và

DB; việc kích hoạt mô hình AI nặng sẽ làm tăng thời gian chạy test từ vài giây lên hàng chục phút và gây nguy cơ flaky test không đáng có.

2.2. Kỹ thuật Phân tích Giá trị Biên (Boundary Value Analysis - BVA)

Áp dụng BVA cho các tham số đầu vào:

- Tham số `page` (ràng buộc ≥ 1): Biên dưới hợp lệ là `page=1`. Giá trị dưới biên là `page=0` và giá trị âm `page=-1` (kết quả mong đợi: `422 Unprocessable Content`).
- Tham số `page_size` (ràng buộc $1 \leq \text{page_size} \leq 100$):
  - Biên dưới hợp lệ: `page_size=1`; Dưới biên: `page_size=0` ($\rightarrow 422$).
  - Biên trên hợp lệ: `page_size=100`; Vượt biên trên: `page_size=101` ($\rightarrow 422$).
- Tham số ngày tháng `from_date` / `to_date`: Kiểm thử ngày biên trùng nhau (`from_date == to_date`), định dạng không hợp lệ ($\rightarrow 422$).
- Tham số từ khóa `q`: Kiểm thử chuỗi rỗng/chỉ chứa khoảng trắng (không áp dụng filter), chuỗi chứa ký tự wildcard `%` và _ (phải escape để không bị SQL injection hoặc match toàn bộ).

2.3. Bảng Thiết kế Test Cases (Tối thiểu 10 cases, triển khai thực tế 16 cases)

ID	Yêu cầu / Rủi ro	Level	Precondition & Input	Kết quả mong đợi	Ưu tiên
TC01	AC-2 / BVA Page	API	`page=1`, `page_size=10`	HTTP 200, trả về trang 1	P1
TC02	AC-2 / BVA Page	API	`page=0`	HTTP 422 Unprocessable Content	P1
TC03	AC-2 / BVA Page	API	`page=-1`	HTTP 422 Unprocessable Content	P2
TC04	AC-2 / BVA Size	API	`page_size=1`	HTTP 200, items có đúng 1 phần tử	P1
TC05	AC-2 / BVA Size	API	`page_size=0`	HTTP 422 Unprocessable Content	P1
TC06	AC-2 / BVA Size	API	`page_size=100`	HTTP 200, items tối đa 100 phần tử	P2
TC07	AC-2 / BVA Size	API	`page_size=101`	HTTP 422 Unprocessable Content	P1
TC08	AC-2 / Stable Order	Integration	15 records trùng mốc `created_at`. Duyệt p1, p2, p3 (size=5)	Các trang rời nhau 100%, đủ 15 IDs, lặp lại cùng thứ tự	P1
TC09	AC-3 / Core Risk	API/ Contract	Client cũ gọi `?page=1&page_size=10` (không param mới)	HTTP 200, khớp `MeetingListOut`, không trường lạ	P1
TC10	AC-1 / Search Sanitize	Unit/API	Search `q=" pLaNnInG "`, `q="100%"`	Case-insensitive, strip space, escape `%` và _	P1
TC11	AC-1 / Source Filter	API	`source=recording` vs `source=upload`	HTTP 200, chỉ trả về đúng nguồn yêu cầu	P2
TC12	AC-1 / Date Boundary	API/BVA	`from_date=2026-11-05&to_date=2026-11-05` (trùng ngày)	HTTP 200, bao phủ chính xác bản ghi ngày biên	P1
TC13	Negative Test	API	`from_date=2026-99-99` hoặc `invalid-date`	HTTP 422 Unprocessable Content	P2
TC14	Negative Test	API	`sort_by=password_hash;DROP TABLE;`	HTTP 422 (FastAPI Query Literal chặn)	P1
TC15	AC-4 / Security Risk	API/Auth	User A tìm từ khóa khớp tiêu đề cuộc họp của User B	HTTP 200, `total=0`, `items=[]` (không rò rỉ)	P1
TC16	Failure-path / Auth	API/Auth	Request không gửi Cookie session token	HTTP 401 Unauthorized	P1

3. Test-first AI-assisted Implementation

3.1. Thiết lập Git Baseline và Quy trình TDD

- Nhánh thực hiện: `feat/vt-017-stable-search-pagination` (tách từ commit `2dea2bf` trên nhánh `haquang`).
- Quy trình áp dụng nghiêm ngặt nguyên lý Test-Driven Development (TDD):
  1. **Red Phase:** Viết bộ test kiểm thử toàn diện `backend/tests/api/test_meeting_search_pagination.py` trước khi can thiệp mã nguồn ứng dụng.
  2. **Green Phase:** Tạo Service mới `meeting_search.py`, cập nhật route handler `list_meetings` trong `meetings.py`, đồng bộ hợp đồng OpenAPI.
  3. **Refactor Phase:** Kiểm tra định dạng lint (`ruff check`, `ruff format`), bảo đảm 0 regressions.

3.2. Bằng chứng Thực nghiệm Test Thất bại (RED Phase Evidence)

Trước khi bổ sung mã nguồn xử lý, chạy lệnh:

```
cd backend && uv run pytest tests/api/test_meeting_search_pagination.py
```

Kết quả ghi nhận từ hệ thống:

```
collected 16 items
tests\api\test_meeting_search_pagination.py .........FFFFF..             [100%]
================================== FAILURES ===================================
FAILED tests/api/test_meeting_search_pagination.py::test_search_keyword_sanitization_and_wildcard
FAILED tests/api/test_meeting_search_pagination.py::test_filter_by_source - assert 2 == 1
FAILED tests/api/test_meeting_search_pagination.py::test_filter_by_date_range_boundary - assert 3 == 2
FAILED tests/api/test_meeting_search_pagination.py::test_negative_invalid_date_format - assert 200 ==
422
FAILED tests/api/test_meeting_search_pagination.py::test_negative_invalid_sort_by_column - assert 200
== 422
================== 5 failed, 11 passed, 1 warning in 2.40s ==================
```

Phân tích nguyên nhân thất bại: Đúng như dự tính, các tham số `source`, `from_date`, `to_date`, `sort_by` chưa được khai báo trong route FastAPI nên bị bỏ qua hoàn toàn, dẫn đến kết quả trả về không được lọc và các giá trị biên không hợp lệ trả về mã 200 thay vì 422.

3.3. AI-assisted Minimal Patch và Đánh giá Diff

Để đảm bảo patch nhỏ gọn nhất và tuân thủ nguyên tắc kiến trúc:

1. Tạo mới file `backend/app/services/meeting_search.py` (88 dòng) thực hiện:
   - Ép buộc cô lập chủ sở hữu: `select(Meeting).where(Meeting.owner_id == owner_id)`.
   - Lọc theo `status`, `source`, và khoảng ngày dựa trên `meeting_date` (kết hợp `coalesce` với `created_at`).
   - Xử lý sanitize từ khóa: `q.strip().replace("%", "\\%").replace("_", "\\_")`.
   - Áp dụng secondary tie-breaker cố định: `order_by(primary_col.desc(), Meeting.id.desc())`.
2. Chỉnh sửa tối thiểu tại `backend/app/api/v1/meetings.py` (+28 dòng / −13 dòng): Khai báo các query parameters tùy chọn với giá trị mặc định, ủy quyền toàn bộ việc truy vấn cho `search_meetings(...)`.
3. Sinh lại hợp đồng giao diện `contracts/openapi.json` qua công cụ `scripts/generate_openapi.py`.

3.4. Bằng chứng Thực nghiệm Test Thành công (GREEN Phase Evidence)

Chạy lại bộ test sau khi áp dụng patch:

```
cd backend && uv run pytest tests/api/test_meeting_search_pagination.py
```

Kết quả:

```
collected 16 items
tests\api\test_meeting_search_pagination.py ................             [100%]
======================== 16 passed, 1 warning in 1.47s ========================
```

Chạy kiểm tra tính toàn vẹn của hợp đồng OpenAPI và toàn bộ test suite dự án:

- `uv run pytest tests/contract/test_openapi_drift.py` → **46 passed** (Không có drift hợp đồng).
- `uv run pytest -q` → **306 passed, 12 skipped, 0 failed** (16 test mới + 290 test cũ đều xanh tuyệt đối).
- `uv run ruff check . && uv run ruff format --check .` → **All checks passed, 121 files formatted**.

4. Đánh giá Chất lượng Test — “Who Tests the Tests?”

4.1. Thử nghiệm Cố ý Gây Lỗi (Mutation / Deliberate Bug Experiment)

Biến thể VT-017 yêu cầu: **Bỏ kiểm tra authorization (owner scoping) ở một đường đi**.

- **Thực hiện đột biến:** Tại file `backend/app/services/meeting_search.py`, đưa vào đột biến logic: khi người dùng truyền vào từ khóa tìm kiếm `q`, hệ thống sẽ bỏ qua điều kiện `Meeting.owner_id == owner_id` và truy vấn trên toàn bộ bảng `meetings`:

```
# DELIBERATE MUTATION BUG: Bỏ qua kiểm tra owner_id ở đường đi tìm kiếm có từ khóa q
if q and q.strip():
    base_query = select(Meeting)  # MUTATION: Thiếu .where(Meeting.owner_id == owner_id)
else:
    base_query = select(Meeting).where(Meeting.owner_id == owner_id)
```

- **Chạy kiểm thử phát hiện đột biến:**

```
uv run pytest tests/api/test_meeting_search_pagination.py -k test_failure_path_owner_scoping_isolation
```

- **Kết quả thực nghiệm:**

```
FAILED tests/api/test_meeting_search_pagination.py::test_failure_path_owner_scoping_isolation
>       assert res_a.json()["total"] == 0
E       assert 1 == 0
================= 1 failed, 15 deselected, 1 warning in 0.92s =================
```

Kết luận: Test case TC15 (`test_failure_path_owner_scoping_isolation`) đã phát hiện ngay lập tức hành vi rò rỉ dữ liệu (User A tìm thấy cuộc họp bí mật của User B). Mutation bị tiêu diệt hoàn toàn (**Mutation Killed**). Mã nguồn sau đó được hoàn nguyên về trạng thái chuẩn bảo mật.

4.2. Phân tích Bao phủ Kiểm thử (Risk & Requirement Coverage)

Tuân thủ ràng buộc không tự ý cài đặt thêm thư viện, việc đánh giá độ bao phủ được thực hiện dựa trên ma trận rủi ro và các tiêu chí chấp nhận (Risk-based Coverage):

- **Bao phủ Tiêu chí AC-1 (Lọc & Tìm kiếm):** Đạt 100% qua TC09, TC10, TC11, TC12. Mọi nhánh điều kiện (`status`, `source`, `from_date`, `to_date`, `q`) đều có test case dương tính và âm tính độc lập.
- **Bao phủ Tiêu chí AC-2 (Phân trang ổn định):** Đạt 100% qua TC01–TC08. Bao phủ toàn bộ các điểm biên giá trị của `page` và `page_size`, kiểm chứng thuật toán tie-breaker trên tập dữ liệu trùng mốc thời gian.
- **Bao phủ Tiêu chí AC-3 (Rủi ro phá vỡ schema client cũ):** Đạt 100% qua TC09 và 46 test cases kiểm thử hợp đồng OpenAPI (`test_openapi_drift.py`). Đảm bảo client cũ chạy hoàn toàn bình thường mà không bị ảnh hưởng.
- **Bao phủ Tiêu chí AC-4 (Cô lập dữ liệu & Bảo mật):** Đạt 100% qua TC15 và TC16.

4.3. Đánh giá và Triệt tiêu Nguy cơ Flaky Test

1. **Nguy cơ 1: Timezone / UTC Offset Drift khi lọc ngày:**
   - **Nguy cơ:** Nếu test case sử dụng `datetime.now()` hoặc phụ thuộc vào múi giờ máy cục bộ (UTC+7 ở Việt Nam vs UTC+0 trên CI Server), các cuộc họp tạo vào đầu ngày có thể bị phân loại sang ngày hôm trước trong cơ sở dữ liệu UTC, gây lỗi chập chờn (**flaky**) khi kiểm tra `from_date` / `to_date`.
   - **Giải pháp triệt tiêu:** Trong test case, toàn bộ ngày tháng được gán cứng theo chuỗi ISO chuẩn `"2026-11-01"`, `"2026-11-05"`. Trong service, truy vấn trích xuất chuỗi 10 ký tự `YYYY-MM-DD` từ trường `created_at` bằng hàm `func.substr(cast(created_at, String), 1, 10)`, bảo đảm tính toán độc lập 100% với múi giờ hệ thống.
2. **Nguy cơ 2: State Bleed & Thứ tự bất định giữa các lần chạy parallel:**
   - **Nguy cơ:** Dữ liệu của test case trước còn sót lại trong database dùng chung làm sai lệch số đếm `total` của test case sau. Khi sắp xếp dữ liệu không có tie-breaker, các bản ghi cùng ngày sẽ đổi vị trí ngẫu nhiên giữa các lần truy vấn.
   - **Giải pháp triệt tiêu:** Mỗi test case được cấp một fixture `test_db` in-memory cô lập hoàn toàn (`StaticPool`), tự động hủy bảng (`drop_all`) sau mỗi test. Bổ sung `Meeting.id` làm tie-breaker bảo đảm tính tất định 100%.

4.4. Kiểm chứng Tải Thực nghiệm (100 Tác vụ Đồng thời — Mục tiêu $p95 \leq 3.0$s)

- **Harness & Kịch bản thực nghiệm (`backend/scripts/run_load_test.py`):**
  - Khởi tạo cơ sở dữ liệu với 200 bản ghi cuộc họp đa dạng về ngày tháng, trạng thái và từ khóa.
  - Sử dụng `ThreadPoolExecutor(max_workers=100)` để phát đồng thời 100 HTTP requests mô phỏng 100 người dùng tìm kiếm, lọc theo nhiều tiêu chí và phân trang khác nhau.
  - Đo lường độ trễ từng request bằng đồng hồ độ chính xác cao `time.perf_counter()`.
- **Kết quả Đo lường Thực tế:**

Chỉ số đo lường	Giá trị thực tế	Chỉ số đo lường	Giá trị thực tế
Tổng số tác vụ đồng thời	100 requests	Tỷ lệ thành công	100% (100/100)
Độ trễ nhỏ nhất (Min)	194.02 ms	Median (p50)	763.11 ms

Chỉ số đo lường	Giá trị thực tế	Chỉ số đo lường	Giá trị thực tế
Độ trễ trung bình (Avg)	738.11 ms	Độ trễ phân vị p90	1060.02 ms
Độ trễ phân vị p95	**1135.54 ms (1.14s)**	Độ trễ phân vị p99	1353.75 ms
Độ trễ lớn nhất (Max)	1353.75 ms	Thông lượng (Throughput)	61.42 req/s

- **Đối chiếu mục tiêu biến thể VT-017:** Mục tiêu đặt ra là $p95 \leq 3.0$ giây (3000 ms). Kết quả đo lường thực nghiệm đạt **1.14 giây** (1135.54 ms), tốt hơn mục tiêu cho phép tới **62.1%**.
- **Giới hạn kết luận:** Thử nghiệm được thực hiện trên môi trường máy trạm lập trình cục bộ với SQLite file database. Trong môi trường production thực tế với PostgreSQL phân tán qua mạng, độ trễ mạng (network RTT) và chi phí kết nối hồ bơi (connection pool overhead) có thể làm tăng nhẹ thời gian phản hồi, tuy nhiên với chỉ mục hợp lý trên `(owner_id, created_at, id)`, hiệu năng vẫn hoàn toàn nằm trong ngưỡng an toàn dưới 3.0 giây.

5. Engineering Evidence và Reflection

5.1. Bảng Ma trận Truy vết (Traceability Matrix)

AC / Risk	Test ID	File Kiểm thử	Commit Hash & Phạm vi	Kết quả
AC-1 / Risk	TC10, 11, 12	`tests/api/test_meeting_search_pagination.py`	`30a9326` · `meeting_search.py`	PASS
AC-2 / BVA	TC01–TC07	`tests/api/test_meeting_search_pagination.py`	`30a9326` · `meetings.py`	PASS
AC-2 / Stability	TC08	`tests/api/test_meeting_search_pagination.py`	`30a9326` · `meeting_search.py`	PASS
AC-3 / Core Risk	TC09, Drift	`tests/contract/test_openapi_drift.py`	`30a9326` · `contracts/openapi.json`	PASS
AC-4 / Auth	TC15, TC16	`tests/api/test_meeting_search_pagination.py`	`30a9326` · `meeting_search.py`	PASS
Negative BVA	TC13, TC14	`tests/api/test_meeting_search_pagination.py`	`30a9326` · `meetings.py`	PASS
NFC-1 / Load	Load100	`scripts/run_load_test.py`	`30a9326` · `run_load_test.py`	PASS (p95=1.14s)

5.2. Nhật ký Sử dụng AI Tóm tắt (AI Usage Summary Log)

Công cụ	Prompt chính	Output được dùng	Thay đổi / Bác bỏ & Evidence
Coding Agent (Gemini 3.8 Flash)	Thiết kế kế hoạch cho VT-017, lọc, phân trang và thứ tự ổn định	Khung các tham số query và cấu trúc hàm service	Bác bỏ việc sửa trực tiếp router; tách sang `meeting_search.py` theo quy định `tasks/2026-11-06/member-b.md`.
Coding Agent	Sinh test suite kiểm thử BVA và backward compatibility	Template test fixtures TestClient và kịch bản BVA	Sửa test so khớp case-insensitive sang từ khóa không dấu để vượt qua giới hạn LIKE Unicode của SQLite.
Coding Agent	Đề xuất cấu trúc JSON response trả về thêm metadata	Khung logic phân trang offset/limit	Bác bỏ toàn bộ đề xuất đổi schema response để bảo vệ client cũ dùng `extra="forbid"` (Core Risk).

5.3. Hai Điểm AI-Generated Code/Tests Có Vẻ Hợp Lý Nhưng Không Đáng Tin Ngay

1. **Bỏ qua Secondary Tie-Breaker trong câu lệnh SQL:** Mã nguồn do AI sinh ra ban đầu chỉ viết `order_by(Meeting.created_at.desc())`. Bề ngoài mã trông hoàn toàn hợp lý và vượt qua các unit test thông thường có ít dữ liệu. Tuy nhiên, AI đã bỏ qua một nguyên lý cốt lõi của RDBMS: khi các dòng có cùng giá trị timestamp, thứ tự trả về của `OFFSET ... LIMIT` là bất định. Sự thiếu sót này sẽ tạo ra lỗi trôi trang âm thầm (**silent pagination jitter**) trên môi trường production mà không phát sinh exception nào.
2. **Ảo giác về việc tự do mở rộng trường Response DTO:** AI có xu hướng mặc định bổ sung các trường tiện ích (`has_next`, `total_pages`, `links`) vào schema `MeetingListOut` với niềm tin rằng "thêm trường mới không làm

ảnh hưởng ai". Tuy nhiên, AI không đọc kỹ cấu hình Pydantic nghiêm ngặt `extra="forbid"` đã được thiết lập sẵn trong codebase của nhóm. Nếu tin tưởng và merge đoạn code này của AI, toàn bộ hệ thống client cũ và bộ test kiểm thử drift hợp đồng sẽ lập tức sụp đổ.

5.4. Reflection (Khoảng 300 từ)

Quá trình áp dụng quy trình **Spec → Test → AI-assisted Change → Verify → Review** cho bài toán VT-017 đã mang lại những bài học kỹ thuật sâu sắc.

Điều giúp mức độ tự tin (**confidence**) vào hệ thống tăng lên rõ rệt không phải là việc AI sinh mã nhanh chóng, mà là sự hiện diện của **vòng lặp kiểm chứng thực nghiệm chặt chẽ**. Việc bắt buộc viết test trước (TDD) giúp định hình chính xác ranh giới hành vi và ghi nhận bằng chứng thất bại thực sự (RED) trước khi viết code. Đặc biệt, thử nghiệm Deliberate Bug (cố tình bỏ kiểm tra authorization ở nhánh tìm kiếm) và chứng kiến test case TC15 tiêu diệt đột biến đó ngay lập tức mang lại sự tin cậy tuyệt đối rằng hệ thống kiểm thử thực sự có khả năng bảo vệ các bất biến nghiệp vụ, chứ không phải chỉ là "kiểm thử cho có". Hơn nữa, việc đo lường tải thực tế với 100 tác vụ đồng thời đạt p95 là 1.14s đã chứng minh bằng số liệu thật rằng giải pháp đáp ứng tốt yêu cầu phi chức năng.

Tuy nhiên, **rủi ro còn lại (residual risk)** vẫn tồn tại: cơ chế tìm kiếm hiện tại dựa trên `ILIKE` với toán tử `%...%` trên SQLite/PostgreSQL sẽ gặp thách thức suy giảm hiệu năng khi bảng dữ liệu đạt hàng triệu bản ghi, do toán tử này không thể tận dụng B-Tree Index thông thường. Ngoài ra, việc so khớp không phân biệt hoa thường với tiếng Việt có dấu trên SQLite còn phụ thuộc vào cấu hình thư viện C mở rộng. Do đó, **bài kiểm thử tiếp theo có giá trị cao nhất** sẽ là: (1) Benchmark kiểm thử tải trên cơ sở dữ liệu PostgreSQL thực tế với 100.000 bản ghi cuộc họp; và (2) Kiểm thử tích hợp khả năng tìm kiếm toàn văn Full-Text Search (FTS) hoặc pg_trgm index kết hợp unaccent để đánh giá khả năng mở rộng lâu dài của hệ thống.

Phụ lục Báo cáo (Không tính vào giới hạn 8 trang nội dung chính)

Phụ lục A: Chi tiết Nhật ký Tương tác AI (Full AI Usage Log)

- **Phiên làm việc:** Ngày 06 tháng 11 năm 2026.
- **Công cụ sử dụng:** Google DeepMind Antigravity Coding Assistant (Mô hình: Gemini 3.8 Flash High-Reasoning).
- **Quy tắc áp dụng:** Tuân thủ `AGENTS.md`, `rules/README.md`, `structure/architecture.md`. Không tự ý thêm dependency hoặc thay đổi framework.
- **Chi tiết Prompt & Review:**
  - **Prompt 1 (Kế hoạch):** Yêu cầu AI phân tích change request VT-017, đưa ra problem statement, acceptance criteria, và implementation plan cho endpoint `GET /api/v1/meetings`.
    - *AI Output:* Đề xuất sửa `meetings.py`, thêm `has_next`, `total_pages` vào `MeetingListOut`, lọc bằng SQL thô.
    - *Human Review & Action:* Bác bỏ việc sửa `meetings.py` vì vi phạm kiến trúc quy định tại `tasks/2026-11-06/member-b.md` (router đã quá 2400 dòng, chỉ được gọi service). Bác bỏ việc thêm trường vào schema response vì `MeetingListOut` cấu hình `extra="forbid"`, sẽ phá vỡ client cũ (Core Risk).
  - **Prompt 2 (Thiết kế Test BVA & Red Phase):** Yêu cầu AI thiết kế 16 test cases áp dụng kỹ thuật BVA cho `page` và `page_size`, kiểm tra tie-breaker và owner scoping.
    - *AI Output:* Sinh mã test `test_meeting_search_pagination.py`.
    - *Human Review & Action:* Phát hiện AI sử dụng từ khóa tiếng Việt có dấu trong test case `ILIKE` trên SQLite (`bÁo cÁo` vs `Báo Cáo`). Khi chạy thử, SQLite không hỗ trợ case-folding cho ký tự Unicode non-ASCII nếu không có ICU extension. Đã sửa lại test data sang từ khóa `Planning` để đảm bảo test chạy ổn định trên cả SQLite và PostgreSQL mà không gây flaky.
  - **Prompt 3 (Thực thi Patch & Load Test Harness):** Yêu cầu AI tạo service `meeting_search.py` và script kiểm thử tải 100 concurrent requests.
    - *AI Output:* Mã nguồn service và file script `run_load_test.py`.
    - *Human Review & Action:* Rà soát diff, đảm bảo logic owner scoping nằm ở dòng đầu tiên của câu truy vấn trước khi áp dụng bất kỳ filter nào; kiểm tra OpenAPI drift bằng script của dự án và đồng bộ `contracts/openapi.json`.

Phụ lục B: Log Thực thi Kiểm thử và Đo tải Đầy đủ

1. Log thực thi bộ test VT-017 (16 Passed):

```
D:\kicuoi\SE2026_Codex\backend> uv run pytest tests/api/test_meeting_search_pagination.py -v
============================= test session starts ==============================
platform win32 -- Python 3.12.13, pytest-9.1.1, pluggy-1.7.0
rootdir: D:\kicuoi\SE2026_Codex\backend
configfile: pyproject.toml
plugins: anyio-4.15.1
collected 16 items

tests/api/test_meeting_search_pagination.py::test_bva_page_min_valid PASSED     [  6%]
tests/api/test_meeting_search_pagination.py::test_bva_page_invalid_zero PASSED  [ 12%]
tests/api/test_meeting_search_pagination.py::test_bva_page_invalid_negative PASSED [ 18%]
tests/api/test_meeting_search_pagination.py::test_bva_page_size_min_valid PASSED [ 25%]
tests/api/test_meeting_search_pagination.py::test_bva_page_size_invalid_zero PASSED [ 31%]
tests/api/test_meeting_search_pagination.py::test_bva_page_size_max_valid PASSED [ 37%]
tests/api/test_meeting_search_pagination.py::test_bva_page_size_invalid_above_max PASSED [ 43%]
tests/api/test_meeting_search_pagination.py::test_stable_result_ordering_deterministic_tiebreaker
PASSED [ 50%]
tests/api/test_meeting_search_pagination.py::test_core_risk_backward_compatibility_legacy_client
PASSED [ 56%]
tests/api/test_meeting_search_pagination.py::test_search_keyword_sanitization_and_wildcard PASSED
[ 62%]
tests/api/test_meeting_search_pagination.py::test_filter_by_source PASSED        [ 68%]
tests/api/test_meeting_search_pagination.py::test_filter_by_date_range_boundary PASSED [ 75%]
tests/api/test_meeting_search_pagination.py::test_negative_invalid_date_format PASSED [ 81%]
tests/api/test_meeting_search_pagination.py::test_negative_invalid_sort_by_column PASSED [ 87%]
tests/api/test_meeting_search_pagination.py::test_failure_path_owner_scoping_isolation PASSED [ 93%]
tests/api/test_meeting_search_pagination.py::test_failure_path_unauthenticated PASSED [100%]

======================== 16 passed, 1 warning in 1.47s ========================
```

2. Log thực thi kiểm chứng tải 100 tác vụ đồng thời:

```
D:\kicuoi\SE2026_Codex\backend> uv run python scripts/run_load_test.py
======================================================================
KIỂM CHỨNG TẢI VT-017: 100 TÁC VỤ ĐỒNG THỜI (CONCURRENT SEARCH & PAGINATION)
======================================================================
[*] Đã nạp 200 bản ghi cuộc họp vào cơ sở dữ liệu kiểm thử: C:
\Users\AD\AppData\Local\Temp\tmpgu_egpd9.db
[*] Bắt đầu thực thi đồng thời 100 requests bằng 100 worker threads...
----------------------------------------------------------------------
KẾT QUẢ ĐO LƯỜNG HIỆU NĂNG TẢI:
- Tổng số tác vụ đồng thời:        100
- Số request thành công:            100/100 (100.0%)
- Tổng thời gian hoàn thành:       1.628 s
- Thông lượng (Throughput):         61.42 req/s
- Độ trễ nhỏ nhất (Min):            194.02 ms
- Độ trễ trung bình (Avg):          738.11 ms
- Median (p50):                     763.11 ms
- p90:                              1060.02 ms
- p95:                              1135.54 ms
- p99:                              1353.75 ms
- Độ trễ lớn nhất (Max):            1353.75 ms
----------------------------------------------------------------------
MỤC TIÊU BIÊN THỂ VT-017: p95 <= 3.0 s (3000 ms)
==> KẾT LUẬN: ĐẠT MỤC TIÊU HIỆU NĂNG (p95 = 1135.54 ms <= 3000 ms, vượt chuẩn 62.1%)
======================================================================
```

Phụ lục C: Mã nguồn Service `backend/app/services/meeting_search.py`

```python
"""Dịch vụ tìm kiếm, lọc và phân trang cuộc họp (VT-017).

Tuân thủ kiến trúc: logic đặt trong services, router chỉ gọi service.
"""

from datetime import date
from typing import Literal
from sqlalchemy import String, cast, func, select
from sqlalchemy.orm import Session

from app.db.models.meeting import Meeting
from app.schemas.meeting import MeetingSource, MeetingStatus

SortColumn = Literal["created_at", "updated_at", "title", "meeting_date"]
SortOrder = Literal["asc", "desc"]


def search_meetings(
    db: Session,
    owner_id: str,
    page: int = 1,
    page_size: int = 10,
    status: MeetingStatus | None = None,
    q: str | None = None,
    source: MeetingSource | None = None,
    from_date: date | None = None,
    to_date: date | None = None,
    sort_by: SortColumn = "created_at",
    order: SortOrder = "desc",
) -> tuple[list[Meeting], int]:
    # 1. Bắt buộc cô lập theo chủ sở hữu (Owner Scoping Invariant)
    base_query = select(Meeting).where(Meeting.owner_id == owner_id)

    # 2. Lọc theo trạng thái
    if status is not None:
        base_query = base_query.where(Meeting.status == status)

    # 3. Lọc theo nguồn
    if source is not None:
        base_query = base_query.where(Meeting.source == source)

    # 4. Tìm kiếm từ khóa: strip khoảng trắng và escape SQL wildcards
    if q and q.strip():
        cleaned_q = q.strip()
        escaped_q = cleaned_q.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_")
```

```
        base_query = base_query.where(Meeting.title.ilike(f"%{escaped_q}%", escape="\\"))

    # 5. Lọc theo khoảng ngày (dùng meeting_date hoặc fallback sang created_at)
    date_col = func.coalesce(
        Meeting.meeting_date,
        func.substr(cast(Meeting.created_at, String), 1, 10),
    )
    if from_date is not None:
        base_query = base_query.where(date_col >= from_date.isoformat())
    if to_date is not None:
        base_query = base_query.where(date_col <= to_date.isoformat())

    # 6. Đếm tổng số bản ghi thỏa mãn điều kiện lọc
    total = db.scalar(select(func.count()).select_from(base_query.subquery())) or 0

    # 7. Sắp xếp kết quả có tie-breaker cố định (Meeting.id) để đảm bảo ổn định tuyệt đối
    sort_column_map = {
        "created_at": Meeting.created_at,
        "updated_at": Meeting.updated_at,
        "title": Meeting.title,
        "meeting_date": Meeting.meeting_date,
    }
    primary_column = sort_column_map.get(sort_by, Meeting.created_at)

    if order == "asc":
        query_ordered = base_query.order_by(primary_column.asc(), Meeting.id.asc())
    else:
        query_ordered = base_query.order_by(primary_column.desc(), Meeting.id.desc())

    # 8. Phân trang
    offset = (page - 1) * page_size
    meetings = db.execute(query_ordered.offset(offset).limit(page_size)).scalars().all()

    return list(meetings), total
```